Supplementary material for: Chance Constraints as Effective Deadlines for Fast Replanning of UAV Fleets Under Uncertainty

Cao-Phuc Ha, Vu Minh Anh Nguyen, Duong Thanh Tran, and Phu-Nguyen Le

Abstract

This document contains the parts of the paper that the main text only summarises: the planning deadlines and the proofs of Propositions 1 and 2, the stopping rule and its two delay costs, the scenario generator and the fitted error models, the parameters of the compared methods and further details of the protocol, the exact-execution benchmark of the stopping rules, the full results of study 1, further results of study 2 per regime and in tier C, the regimes of study 3, the reliability of the error model in study 4, the re-scoring that motivated study 5, the calibration of the predicted probability in studies 4-7, the sensitivity of all tier B tests to the resampling unit and further results of studies 2-7. Section, equation and table numbers without the prefix S refer to the main paper.

S1 Planning deadlines and proofs

This section states the planning deadlines of Section 4 of the main paper in full and proves Propositions 1 and 2. For task j, f_j is the planned finish time, d_j the deadline, w_j the reward, $h_j = f_j - t_0$ the lead time and $F_j = f_j + e_j$ the actual finish time, with $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$ and m, s as in Eq. (1). The effective deadline of Eq. (2) at safety factor z is written $\tilde{d}_j(z)$, and $\tilde{d}_j(-\infty)$ is read as d_j.

S1.1 Planning deadlines

Let $\tilde{H}$ be the set of tasks that no vehicle can finish by $\tilde{d}_j(z)$ on a direct flight, and H the set of tasks that no vehicle can finish by d_j. The nominal fallback plans against

$$d'_j = \begin{cases} d_j, & j \in \tilde{H} \setminus H, \\ \tilde{d}_j(z), & \text{otherwise.} \end{cases} \tag{S1}$$

For the cost-aware fallback, let $\theta_j = c_j/(w_j + c_j)$ be the break-even probability of Eq. (3), $z_j = \Phi^{-1}(\theta_j)$ with $z_j = -\infty$ for $c_j = 0$, and $z_{p,j} = \max(z, z_j)$. With $\tilde{H}_p$ the set of tasks that no vehicle can finish by $\tilde{d}_j(z_{p,j})$ on a direct flight, the planning deadline is

$$d'_j = \begin{cases} \min\{d_j, \tilde{d}_j(z_j)\}, & j \in \tilde{H}_p \setminus H, \\ \tilde{d}_j(z_{p,j}), & \text{otherwise.} \end{cases} \tag{S2}$$

A task that cannot meet this deadline either is given up. Finding $\tilde{H}_p$ and H requires one direct-flight check per task and vehicle. Where $1 + b_1 + z_j s_1 \le 0.05$, the effective deadline is not defined and the nominal deadline is used, without the guarantee of part (iii) of Proposition 2. With the error models of this paper, this happens only for $\theta_j < 0.03$, which none of the studied late costs produces. Every planner caps the effective deadline at d_j, so that no task is planned beyond its nominal deadline.

S1.2 Proposition 1

In full, Proposition 1 states the following. Let $1 + b_1 + z s_1 > 0$ and $s_0, s_1 \ge 0$. (i) A finish time satisfies $f_j + m(f_j - t_0) + z\, s(f_j - t_0) \le d_j$ if and only if $f_j \le \tilde{d}_j(z)$. (ii) Under the error model, a task planned to finish at $f_j \le \tilde{d}_j(z)$ meets its deadline with probability at least $\Phi(z)$ if its route starts as planned. (iii) With $b_0 = b_1 = s_0 = s_1 = 0$, $\tilde{d}_j(z) = d_j$ and the planning model is the nominal one.

Proof. Substituting the error model gives $f_j(1 + b_1 + z s_1) \le d_j - b_0 - z s_0 + (b_1 + z s_1) t_0$, and dividing by the positive factor gives (i). For (ii), $F_j = f_j + e_j$ with $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$. By (i), $d_j - f_j - m(h_j) \ge z\, s(h_j)$. If $s(h_j) > 0$, then $\Pr[F_j \le d_j] = \Phi\big((d_j - f_j - m(h_j))/s(h_j)\big) \ge \Phi(z)$. If $s(h_j) = 0$, which happens only for $s_0 = 0$ and $h_j = 0$, the error equals $m(h_j)$ with certainty, $F_j \le d_j$ holds, and the probability is one. Part (iii) is immediate. ∎

The cap at d_j matters only where $\tilde{d}_j(z) > d_j$, which happens when $z < 0$ or the mean error is negative. It then adds the nominal deadline to the constraint, which becomes tighter, so part (ii) still holds, whereas the equivalence of part (i) refers to the uncapped deadline.

S1.3 Proposition 2

In full, Proposition 2 states the following. Let $s_0, s_1 \geq 0$, $1 + b_1 + z s_1 > 0$ and $1 + b_1 + z_j s_1 > 0$ for every finite z_j. (i) If $c_j = 0$ for every task, Eq. (S2) is the nominal fallback of Eq. (S1). (ii) A task with $\theta_j \geq \Phi(z)$ never falls back, and it is planned against $\tilde{d}_j(z_j)$. (iii) Under the error model, every task planned to finish by d'_j has an expected contribution $w_j \Pr[F_j \leq d_j] - c_j \Pr[F_j > d_j] \geq 0$ if its route starts as planned. (iv) If every c_j is non-decreasing in λ, every planning deadline d'_j is non-increasing in λ.

Proof. (i) $z_j = -\infty$ gives $z_{p,j} = z$ and $\min\{d_j, \tilde{d}_j(-\infty)\} = d_j$. (ii) $z_j \geq z$ holds if and only if $\theta_j \geq \Phi(z)$. Then $z_{p,j} = z_j$, and a task that cannot meet $\tilde{d}_j(z_{p,j})$ cannot meet $\min\{d_j, \tilde{d}_j(z_j)\}$ either, so it is given up. (iii) A protected task meets its deadline with probability at least $\Phi(z_{p,j}) \geq \Phi(z_j)$, and a fallback task with probability at least $\Phi(z_j)$, both by Proposition 1, whose part (ii) also holds with the cap at d_j. Hence $p \geq \theta_j$, and the expected contribution $(w_j + c_j)p - c_j$ is not negative. (iv) θ_j and z_j do not decrease with λ, and with $s_0, s_1 \geq 0$, $\tilde{d}_j(z)$ does not increase with z. A fallback task stays one when λ grows, because $z_{p,j}$ does not decrease. A protected task can become a fallback task only while $z_j < z$, and then $z_{p,j} = z$ does not change. In each case the deadline does not increase. ∎

Part (iii) is a local criterion. It prices each attempt on its own and ignores the delay that a fallback task causes to the tasks after it on the same route, as well as the reassignments it may trigger, so it does not guarantee that attempting the task raises the utility of the whole plan.

S2 Stopping rule and delay costs

S2.1 Learned stopping rule

The objective of a plan π in Section 3 of the main paper is $J(\pi) = \alpha R(\pi) - \gamma E(\pi) - \delta C(\pi)$, where R is the reward of the tasks served on time, E the flown distance in kilometres and C the number of tasks that move to another UAV, with $\alpha = 1$, $\gamma = 0.05$ per kilometre and $\delta = 0.3$. W denotes the total reward of the open tasks. Every 5 ms, the stopping rule compares the predicted gain of searching for another $H = 10$ ms with the cost of doing so. With $\widehat{\Delta J}$ as the predicted gain, the search stops at time τ if

$$\widehat{\Delta J} \;\leq\; C_{\text{delay}}(\tau) + \mu W H. \tag{S3}$$

The gain is predicted from the search history by a Poisson generalised linear model with a log link [1] and a small L_2 penalty. Its eleven features describe the elapsed time, the number of iterations, the recent improvements over 5 ms, over 20 ms and over the second half of the elapsed time, the time since the last improvement, the total improvement over the repaired plan, the unassigned share of the reward, the numbers of tasks and UAVs, and the iteration rate. The target is the improvement in the next 10 ms in percent of W. The model is fitted on 500 ms search trajectories of development scenarios only, and a prediction costs one dot product. The search loop enforces the hard time budget B independently of the stopping rule. The price of computation is $\mu = 10^{-5}$ per millisecond.

S2.2 Deterministic delay cost

The delay cost is computed from the best plan found so far. A late start of a route is partly absorbed by the waiting times along it, in the same way as the forward time slack of routing with time windows [2]. The tolerance tol_j of a task is the margin between its finish time and its deadline, plus all waiting times on the route up to this task. A task is lost when the start shift $\sigma_j(\tau)$ of its route exceeds its tolerance. The start shift is zero for a UAV that is still busy, and it equals the waiting time until the publication for an idle UAV. With $v_j = \alpha w_j - \delta \mathbb{1}[j \text{ changes UAV}]$ as the value of task j in the objective and $\Lambda(\tau) = \sum_j v_j \mathbb{1}[\sigma_j(\tau) > \text{tol}_j]$, the delay cost of searching H milliseconds longer is $C_{\text{delay}}(\tau) = \Lambda(\tau + H) - \Lambda(\tau)$.

Proposition S1. For a feasible plan executed exactly, (i) $\Lambda(\tau)$ is the value lost through missed deadlines if every UAV follows its route, (ii) the loss of the objective is at most $\Lambda(\tau)$ if late tasks are skipped, and (iii) $\Lambda(\tau)$ does not decrease with τ and is zero while all UAVs are busy with their committed tasks. Hence C_{delay} is non-negative and vanishes while no UAV is waiting.

Proof. A route that starts later by some shift reaches its first task later by the same amount. Each waiting time absorbs part of the delay, so the delay at the end of a task equals the start shift minus all earlier waiting times, if this difference is positive. The task therefore misses its deadline exactly when the start shift exceeds its tolerance. Waiting consumes no energy, which proves (i). If late tasks are skipped, the triangle inequality ensures that every later task is reached no later and with no more energy. All tasks that are on time in (i) remain on time, and the flown distance, hence the energy term γE, does not grow. The objective therefore loses the values v_j of the skipped tasks and nothing else, which is at most $\Lambda(\tau)$ and proves (ii). Part (iii) holds because the start shift does not decrease with τ and every tolerance is non-negative in a feasible plan. ∎

Table S1: Scenario generator parameters (distances in m, times in s, energy in metres of flight).

Map side / horizon	10 000 / 1 800	Service time	$U[30, 90]$
UAV speed	$U[10, 16]$ m/s	Time-window slack	$U[180, 600]$
Energy capacity	$U[18, 28]$ km	Release delay	$U[0, 120]$
Energy reserve	10% of capacity	Rewards 1, 2, 3, 5	probabilities .4, .3, .2, .1
Hover cost	6 per second	Hotspots	3, std. 600, 50% of tasks
Tasks known at start	50%	Scales (UAVs/tasks)	5/25, 10/50, 20/140, 40/280

S2.3 Execution-aware delay cost

Under uncertain execution, task j is lost when $\sigma_j(\tau) + e_j > \mathrm{tol}_j$, where e_j is its finish-time error. The execution-aware cost replaces the indicator in Λ by the probability of this event. The error of task j is modelled as $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$, where $h_j = f_j - t_0$ is its lead time, the nominal finish time minus the time of the planning snapshot. The mean and the spread are linear in the lead time,

$$m(h) = b_0 + b_1 h, \qquad s(h) = s_0 + s_1 h, \tag{S4}$$

because errors accumulate along a route. The expected dissemination latency $\bar{L}$ is added to the start shift. The expected loss and the execution-aware delay cost are

$$\Lambda_{\mathrm{R}}(\tau) = \sum_j v_j \, \Phi\!\left(\frac{\sigma_j(\tau) + \bar{L} + m(h_j) - \mathrm{tol}_j}{s(h_j)} \right), \qquad C_{\mathrm{R}}(\tau) = \Lambda_{\mathrm{R}}(\tau + H) - \Lambda_{\mathrm{R}}(\tau), \tag{S5}$$

where Φ is the standard normal distribution function. Search and predictor are unchanged, and C_{R} replaces C_{delay} in Eq. (S3).

Proposition S2. (i) With $b_0 = b_1 = s_0 = s_1 = 0$ and $\bar{L} = 0$, $\Lambda_{\mathrm{R}} = \Lambda$, and the stopping rule takes exactly the decisions of the deterministic one. (ii) $\Lambda_{\mathrm{R}}(\tau)$ does not decrease with τ, so $C_{\mathrm{R}} \geq 0$. (iii) Under the error model, $\Lambda_{\mathrm{R}}(\tau)$ is the expected value of the tasks whose deadline is missed if every UAV follows its route.

Proof. With zero spread, Φ is read as the indicator of a positive argument, which gives (i). The start shift does not decrease with τ and Φ is non-decreasing, which gives (ii). Part (iii) follows from the proof of Proposition S1, because task j misses its deadline exactly when its delay at the finish, $\sigma_j + e_j$ minus the absorbed waiting time, is positive, and the expectation of the indicator of this event is the probability in Eq. (S5). ∎

The parameters of Eq. (S4) are fitted on development missions. The mean is estimated by least squares, and the spread by least squares on the absolute residuals scaled by $\sqrt{\pi/2}$, which is less sensitive to the heavy tail of the errors than squared residuals. The latency $\bar{L}$ is the mean of the latency distribution in tier B and the measured mean plan-acceptance latency in tier C.

S3 Scenario generator, error models and exact-execution snapshots

S3.1 Scenarios

Missions are created by a seeded generator with the parameters in Table S1. The map is a square with a side of 10 km and the depot at its centre, and half of the tasks are clustered around three hotspots. Half of the tasks are known at the start, and the others arrive in batches. Three fleet scales with 5, 10 and 20 UAVs and 25, 50 and 140 tasks are combined with three event profiles. The arrivals profile contains six arrival batches, the failures profile three batches and two UAV failures, and the deadlines profile three batches and four tightened deadlines. A fourth scale with 40 UAVs and 280 tasks is used for scalability only.

S3.2 Static snapshots with exact execution

The stopping rules are first compared without execution uncertainty. Each mission is simulated once with a deterministic reference planner, and every replanning call after the start is stored with its previous plan. All methods are evaluated on these snapshots, and regret is measured against the best known objective of each snapshot, which includes a 3 s ALNS run and, for the small and medium scales, the OR-Tools routing solver [3] with guided local search [4]. Because all ALNS variants follow the same search trajectory, whole families of each rule are evaluated exactly offline, with 12 fixed budgets, 10 stagnation patience values and 9 compute prices, and each baseline is compared with AnytimeSwarm at the same mean computation. This benchmark uses its own planned test seeds 2000-2029 (1562 snapshots of 270 missions), and it was reproduced on the final software platform before the execution-aware study (Section S5.1).

Table S2: Finish-time error models fitted on development missions.

Tier	Level	n	b_0	b_1	s_0	s_1
B	low	5399	8.91	−0.0044	0.00	0.0904
B	mid	4991	16.84	0.0354	1.58	0.1951
B	high	4253	20.79	0.2923	0.00	0.6389
C	low	90	−1.23	0.0281	1.30	0.0545
C	mid	89	2.36	0.0297	0.00	0.2303

Mean $m(h) = b_0 + b_1 h$ and spread $s(h) = s_0 + s_1 h$, in seconds of the respective tier; n: executed tasks.

S4 Methods, protocol and further results

S4.1 Error structure

The comparison of spread models in Section 4 of the main paper uses 4 982 and 4 231 pairs of lead time and error from 90 development missions per disturbance level. A hybrid model $s(h) = \sqrt{q_0 + q_1 h + q_2 h^2}$ also fitted worse than the linear spread, by 72 and 117 points of the Akaike information criterion. The flight-time error of a single leg had a coefficient of variation of 0.39 and 1.15 under mid and high disturbances, and missions explained only 5% and 7% of the residual variance.

S4.2 Parameters of the compared methods

The performance profile rule predicts the gain of further search from a static profile over the elapsed time and the number of open tasks. The online prediction rule follows Svegliato et al. [5], but fits the saturating curve $q(t) = a - b\,e^{-ct}$ to the best objective found so far at every check, in place of the arctangent or logarithmic models of the original. Both use the delay cost and the price of computation of AnytimeSwarm-R. The size-scaled budget allows 1 ms per open task, and the stagnation rule stops after 20% of the budget without improvement. The proportional buffer accepts a planned finish f_j if $f_j + c\,h_j \le d_j$. Moment propagation models the flight time of a leg of nominal duration t as normal with mean $(1 + a)t + c_0$ and standard deviation $c_v t$, and the service time with mean $(1 + a_s)s$ and standard deviation $c_s s$. Legs are independent, the start of service $\max(A_j, r_j)$ is approximated by Clark's moments [6], and a finish is accepted if $\mathrm{E}[F_j] + z\,\mathrm{sd}[F_j] \le d_j$. The leg parameters are fitted on the legs of the development flights. Because the variance of one long leg can exceed that of the two legs it replaces, removing a task can make a route infeasible under this model, so every route is filtered once more before publication. With the cost-aware fallback, moment propagation accepts a given-up task if its nominal finish meets the deadline and $\mathrm{E}[F_j] + z_j\,\mathrm{sd}[F_j] \le d_j$. Online calibration starts from one error model fitted on both disturbance levels, with the weight of n_0 pairs, and refits Eq. (1) at every call with the pairs of the tasks completed so far, so it needs neither the disturbance level nor a level-specific safety factor. The selected values are $c = 0.45$ and 0.6, $z = 1$ and 0.5 for moment propagation, and $n_0 = 50$ with $z = 0.5$ for online calibration, under mid and high disturbances. On development missions, the three static models differed by less than 0.2 pp. CBBA bids the largest increase of the route value over all feasible insertion positions, plans from scratch at every call and stops at convergence or at the hard budget. Without the bid warping of Johnson et al. [7], it did not converge on some development snapshots. Within LTCC, the stopping rule uses a zero mean error, because the effective deadlines already contain it.

S4.3 Tuned competitors of studies 5 and 7

In study 5, the proportional buffer was tuned with $c \in \{0.1, 0.2, 0.3, 0.45, 0.6, 0.75, 0.9, 1.2\}$, with or without the nominal fallback, and LTCC without fallback with $z \in \{-0.5, 0, 0.5, 1, 1.5\}$, for every $\lambda \in \{0.5, 1, 2\}$ and disturbance level. A grid that first ended at $c = 0.3$ and $z = 0$ was extended after both values were selected on its edge, and the selection did not change. For $\lambda = 1$, the selected buffer was $c = 0.3$, no fallback under mid and $c = 0.45$, no fallback under high disturbances, and LTCC without fallback had $z = 1$ and $z = 0.5$. Studies 5 and 6 ran the cost-aware fallback with the proportional cost model, $\theta_j = \lambda/(1 + \lambda)$, but scored the runs with u_λ, which charges every late arrival a fixed cost; study 7a removes this mismatch. The fixed-cost rule of study 7a uses the mean reward $\bar{w} = 2.1$ of the generator, because the total reward of a mission is not known while tasks are still arriving, whereas u_λ charges the realised mean reward R/n of the mission. Over the held-out missions, R/n was 2.10 on average, with a coefficient of variation of 8% and a range of 1.64 to 2.76, so study 7a is also reported with $u'_\lambda = U - \lambda \bar{w}\, n_{\mathrm{late}}/R$. In study 7a, the development selection of the tuned buffer differed between u_λ and u^w_λ only at $\lambda = 1$ under high disturbances, with $c = 0.45$ for u_1 and $c = 0.6$ for u^w_1, both without fallback.

Study 7b keeps the error model, the safety factor and every tuned parameter of the calibrated high level. Speed and service errors follow a Student-t law with three degrees of freedom at the same standard deviation; rare long delays

add, with probability 0.1, an exponential delay with a mean of 90 s to a leg, as a detour or a holding pattern would; a persistent speed bias per UAV and mission with a standard deviation of 8% correlates the errors along a route; and a Pareto dissemination delay with the median of the lognormal one and a tail index of 1.5 replaces the lognormal delay. It compares the nominal model, the constant and the proportional buffer, LTCC with the nominal and with the cost-aware fallback ($\lambda = 1$, both cost models) and moment propagation with the cost-aware fallback.

S4.4 Protocol

Studies 2, 3, 4, 5 and 6 used the held-out seeds 5000-5029, 7000-7029, 8000-8029, 10000-10029 and 12000-12029, and studies 7a and 7b used 13000-13029 and 14000-14029. The tier C part of study 2 flew 30 missions with mid disturbances and a latency factor of 50 (seeds 6000-6009), and study 4C used seeds 9000-9011. The protocols were not lodged with an external registry. They are available with the code, together with the commit of every freeze and every deviation. A tier C run in which a vehicle does not take off in time is repeated once, like a run with an invalid real-time factor. Tier B runs four missions in parallel on an Intel Core i7-11800H laptop, with the methods of a configuration run in turn. A separately written validator found no infeasible plan among 4 779 development plans. Because the stopping rules read a wall clock, the mean utility of a method varied by up to 0.12 pp between two runs of the same missions. Equivalence (H3) was tested with two one-sided tests.

S4.5 Further results of studies 2 to 4

In study 2, the 90% interval of the stopping rule minus a fixed 40 ms was +0.22 [+0.09, +0.35] pp, inside the equivalence margin of ±0.5 pp. LTCC without fallback served 5.72 tasks per mission after their deadline, against 11.22 for ALNS with 40 ms. In the tier C part of study 2, the Holm-adjusted p of H4 was 0.942, and about 118 missions per method would have been needed to confirm the difference that tier B predicted. In study 4, the interval of LTCC minus the proportional buffer was −0.72 to −0.05 pp (Holm-adjusted $p = 0.050$) and that of LTCC minus moment propagation −0.62 to 0.12 pp. The loss of moment propagation without fallback had an interval of 9.04 to 10.29 pp. LTCC ran 13.35 search iterations per millisecond and moment propagation 12.22. Online calibration served 0.30 pp less than the model with a known disturbance level, and the learned stopping rule added 0.34 pp. In study 4C, the p-value of H14 was 0.699. The advantage over the nominal model that sized the study was 5.52 pp on development seeds. Over the full tier B grid, the advantage in this regime was 3.56 pp, which would need about 51 missions per method to be confirmed in tier C. The interim analysis did not cross the O'Brien-Fleming bound. The illustrative flights of Fig. 4 ran with the graphical interface, at a real-time factor of 1.22 to 1.34.

S4.6 Further results of studies 5 and 6

In study 5, the advantage of the cost-aware fallback at $\lambda = 1$ came from high disturbances, where it served 2.10 and 3.49 pp more than the tuned proportional buffer and the tuned safety factor. Under mid disturbances, the methods were on par, and with the original windows both tuned competitors were better, by 2.08 and 1.68 pp. At $\lambda = 0.5$, the interval against the tuned buffer was −0.22 to 0.64 pp. At $\lambda = 2$, the cost-aware fallback stayed 7.25 pp above the nominal fallback and matched the tuned safety factor, while the tuned buffer was better (−1.58 to −0.47 pp) and moment propagation without fallback by 1.10 pp. Tuned for $\lambda = 1$, the proportional buffer stayed 1.46 pp behind the cost-aware fallback at $\lambda = 0.5$. A wrong late cost also costs the cost-aware fallback, 1.22 pp when it was run with $\lambda = 1$ and scored at $\lambda = 2$, and when a late arrival costs nothing, the rule run with $\lambda = 1$ served 1.20 pp less utility than the nominal fallback.

In study 6, CBBA on the nominal model reached almost the utility of ALNS (−1.48 to −0.51 pp), and LTCC served 6.72 pp more than CBBA with the event trigger at $\lambda = 0$. CBBA served 10.57 tasks late per mission and LTCC 4.91. Under high disturbances with halved windows at $\lambda = 0$, LTCC and CBBA were on par (−0.24 [−1.29, +0.82] pp), because a fallback for $\lambda = 1$ gives up tasks that a late arrival would not penalise at $\lambda = 0$. Within moment propagation, the cost-aware fallback lost 2.96 pp against the nominal fallback at $\lambda = 0$ and gained 8.38 pp at $\lambda = 2$. The interval of the closed form minus moment propagation, both with the cost-aware fallback, was −1.31 to −0.24 pp at $\lambda = 1$, and moment propagation ran 433.2 search iterations per call against 467.4.

S4.7 Further results of study 7

In study 7a, the fixed-cost rule scored 45.11% with u_1, the proportional rule 44.87% and the tuned buffer 43.78%; the Holm-adjusted p of H24 was 0.286. H26 had a Holm-adjusted p of 0.022 for non-inferiority and $p = 0.499$ for superiority. The fixed-cost rule scored +0.85 [+0.32, +1.39] pp above the proportional rule with u_2, and the proportional rule scored 0.86 and 1.25 pp above the fixed-cost rule with u_1^w and u_2^w. With u_2, the fixed-cost rule minus the tuned buffer was +0.10 [−0.63, +0.82] pp, whereas the proportional rule stayed behind it (−0.75 [−1.38, −0.12] pp). With u_2^w, the proportional rule stayed 1.36 pp behind the buffer tuned for that score. With u_1', the differences of H24 and

Table S3: Regret at equal computation per fleet scale.

		Regret at equal computation (%)			Computation to match (ms)	
Scale	Time (ms)	Proposed	Fixed	Stagnation	Fixed	Stagnation
Small (5/25)	18	0.006	0.030†	0.029†	50 (2.7×)	51 (2.8×)
Medium (10/50)	25	0.030	0.062†	0.045†	49 (2.0×)	39 (1.5×)
Large (20/140)	81	0.213	0.290†	0.274†	172 (2.1×)	135 (1.7×)
40 UAVs (40/280)	151	0.205	0.256†	0.234†	299 (2.0×)	258 (1.7×)
Small to large	42	0.083	0.141†	0.127†	143 (3.4×)	99 (2.4×)

Fleet scale as UAVs/tasks. Baselines run for the mean time of AnytimeSwarm on the same scale; †: 95% interval of the paired difference below zero. Right: computation a baseline needs to match AnytimeSwarm.

H25 were +0.26 [−0.17, +0.69] and +1.35 [+0.79, +1.91] pp. In study 7b, the nominal model served 42.28% under the calibrated errors and between 40.09% with rare long delays and 42.67% with Student-t errors. The cost-aware fallback scored 2.94 to 3.64 pp more than the proportional buffer of study 4, and rare long delays raised the mean finish-time error of LTCC from 78.8 to 88.3 s.

S5 Exact execution: stopping rules at equal computation

On the 1562 test snapshots, regret-2 repair finished within about one millisecond but left a regret of 0.98%. With a budget of 250 ms, ALNS with the full budget reached 0.06%, whereas AnytimeSwarm reached 0.08% with 16% of the budget, that is, with 84% less computation. Because the stopping rules use different amounts of computation, they are best compared at equal computation. Table S3 makes this comparison for each fleet scale, with the fixed budget and the stagnation rule evaluated at the mean computation that AnytimeSwarm used on the same scale. AnytimeSwarm reached a lower regret on every scale, and all paired intervals lie below zero. The reduction was 33% to 52% on the medium scale, 22% to 27% on the large scale and 12% to 20% with 40 UAVs, a scale that was never used for training. Read in the other direction, the two baselines needed between 1.5 and 2.8 times the computation of AnytimeSwarm to reach its regret. Over all scales, AnytimeSwarm reached 0.083% against 0.112% to 0.130% for four baselines that include a fixed budget and a stagnation patience tuned per fleet scale on development data.

Fig. S1 shows where this advantage comes from. On the medium and large scales, the quality-compute frontier of AnytimeSwarm lies below those of fixed budgets and stagnation stopping over the range of practical computation (Fig. S1a-c). Grouped by the improvement that a 500 ms search can still find (Fig. S1d), AnytimeSwarm spent 18.2 ms on snapshots that cannot be improved and 140.0 ms on those with the largest available improvement. Only 29% of its computation went to snapshots without any possible gain, compared with more than half for the baselines. The learned predictor is essential for this allocation. It reached an area under the ROC curve [8] of 0.926 for the event that the next 10 ms bring an improvement, compared with 0.757 for an extrapolation of the recent improvement rate.

S5.1 Reproduction on the evaluation platform

The exact-execution benchmark was produced on Windows. Before the execution-aware study, its development pipeline was re-run on the platform of all later experiments (Ubuntu 24.04 under WSL2, identical Python and package versions). The deterministic part was reproduced bit for bit, except for differences of one unit in the last place of some energy values, which stem from the floating-point libraries of the two operating systems. The search ran 21% more iterations per millisecond on the new platform. The improvement predictor, re-trained on the new timing traces, reached the same compute price by the rule fixed in the protocol, and the conclusions of the equal-computation comparison agreed in sign and significance in 24 of 24 comparisons. The frozen predictor was used unchanged in all later experiments.

S6 Study 1: execution-aware stopping on the nominal planning model

Study 1 compares AnytimeSwarm-R with the stopping rules of Table 3 on the nominal planning model, on held-out seeds 3000-3029 in tier B (270 missions over three scales and three profiles, and 90 missions with 40 UAVs) and 4000-4009 in tier C (30 missions per method). Its primary matrix crosses two execution levels (low, mid), two latency levels (none, high), two latency factors ($\kappa_B = 10, 50$) and two triggers (event, periodic every 60 s). Its primary hypothesis is that AnytimeSwarm-R serves more reward than ALNS with 40 ms, AnytimeSwarm and the stagnation rule.

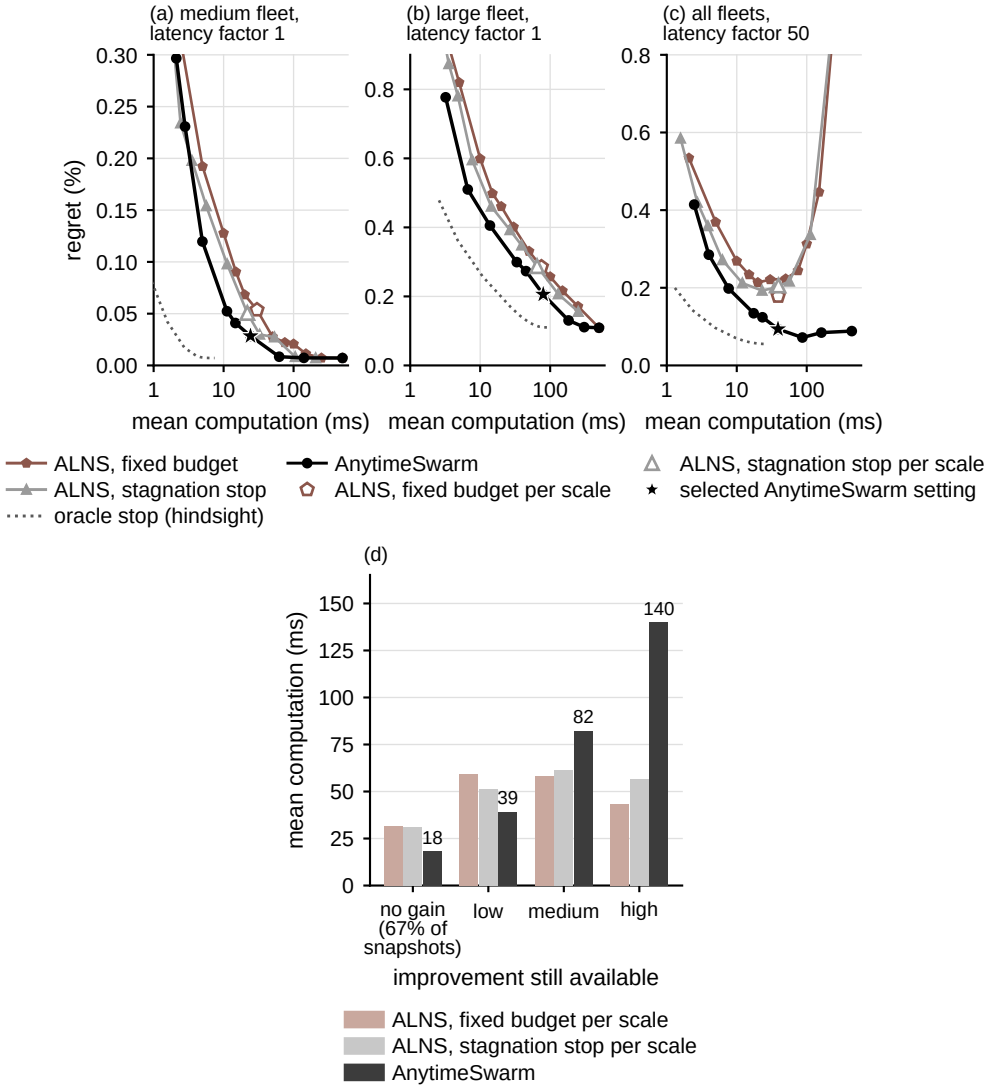


Figure S1: Exact execution on test seeds 2000-2029. (a)-(c) Quality-compute frontiers on identical search trajectories. (d) Mean computation per snapshot, grouped by the improvement that a 500 ms search can still find.

S6.1 Tier B primary matrix

The primary matrix comprises 34 560 mission runs, that is, 270 held-out missions in 16 configurations with eight methods. Table S4 lists utility, lost tasks and computation per method, and Fig. S2 relates utility to computation and shows the difference in every configuration.

Search pays off clearly under uncertain execution. Regret-2 repair reached a utility of 68.86%, and AnytimeSwarm-R reached 72.25%, which is +3.39 percentage points (pp) or 4.9% more. The full budget of 250 ms was the weakest search method, with 71.48%, and AnytimeSwarm-R served +0.77 [+0.31, +1.25] pp more reward than it. The loss of the long budget stems from late plans. A task counts as lost to delay when it is part of the computed plan but can no longer be served on time when the plan is released. With $\kappa_B = 50$, the full budget lost 5.36 tasks per mission in this way, against 1.28 for AnytimeSwarm-R, that is, 76% fewer. Without its delay cost, AnytimeSwarm lost 3.50 tasks per mission in this way, but its utility was not lower (72.13% against 72.04%). The delay cost thus prevents losses from late plans without a measurable gain in utility on this matrix.

Among the remaining search methods, the differences are small. All seven search methods lie within 0.77 pp of each other. AnytimeSwarm-R reached the highest mean utility with the least computation of the adaptive rules, 41.1 ms per call against 48.2 ms for AnytimeSwarm and 59.9 ms for the stagnation rule. The reduction of 15% relative to AnytimeSwarm is significant (−7.1 [−7.7, −6.5] ms per call), so the second hypothesis holds. The primary hypothesis does not hold at the level fixed in the protocol. The pooled advantage of AnytimeSwarm-R was +0.32 [+0.01, +0.63] pp over ALNS with 40 ms, +0.21 [−0.08, +0.52] pp over AnytimeSwarm and +0.28 [−0.11, +0.67] pp over the stagnation rule, with Holm-adjusted p-values of 0.128, 0.308 and 0.308. Only the first interval excludes zero before correction. Per configuration and without correction for multiple testing, AnytimeSwarm-R was better than ALNS with 40 ms in 2 of

Table S4: Study 1: results of the tier B primary matrix.

Method	Utility (%)	Late	Skipped	Calls	ms/call	AnytimeSwarm-R minus method (pp)
Regret-2 repair	68.86	6.64	18.35	15.6	1.0	+3.39 [+2.76, +4.04]
ALNS, 250 ms	71.48	6.15	20.58	15.5	250.1	+0.77 [+0.31, +1.25]
ALNS, 40 ms	71.93	6.65	20.97	15.5	40.1	+0.32 [+0.01, +0.63], p_{Holm} 0.128
ALNS, stagnation	71.97	6.48	20.92	15.5	59.9	+0.28 [−0.11, +0.67], p_{Holm} 0.308
AnytimeSwarm w/o delay cost	72.13	6.35	20.49	15.5	54.1	+0.12 [−0.20, +0.45]
AnytimeSwarm	72.04	6.64	21.32	15.5	48.2	+0.21 [−0.08, +0.52], p_{Holm} 0.308
AnytimeSwarm-R, latency only	72.02	6.65	21.31	15.5	48.2	+0.23 [−0.06, +0.54]
AnytimeSwarm-R	72.25	6.56	21.15	15.5	41.1	-

270 held-out missions in 16 configurations. Utility and lost tasks per mission, mean planning time per call, and pooled paired difference of AnytimeSwarm-R minus each method (pp, 95% bootstrap interval; Holm-adjusted p for the three primary comparisons).

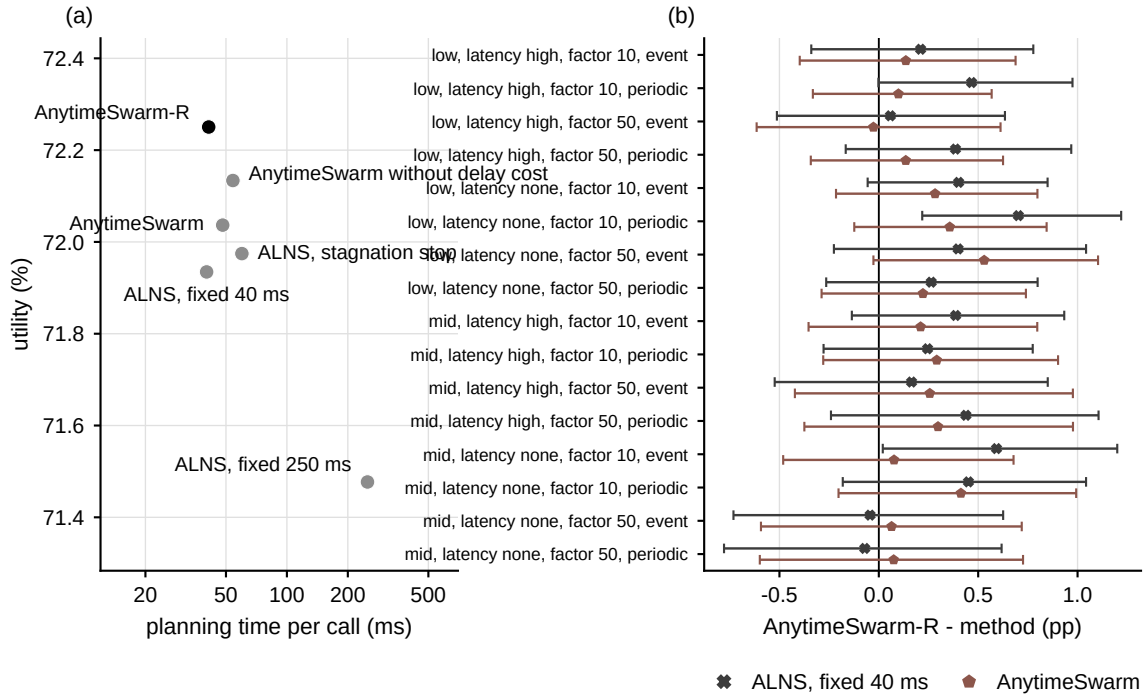


Figure S2: Study 1, tier B. (a) Mean utility against mean planning time per call; regret-2 repair (68.86%, 1 ms) is not shown. (b) AnytimeSwarm-R minus ALNS with 40 ms and minus AnytimeSwarm per configuration, with 95% intervals over 270 missions.

the 16 configurations, better than the stagnation rule in 1 and never different from AnytimeSwarm, which is consistent with chance (Fig. S2b). The ablation that keeps only the latency term reached 72.02%, close to AnytimeSwarm. Together with the misspecification runs below, this suggests that a non-zero error spread mainly makes the rule stop earlier, rather than that the calibration of the spread matters.

The fixed budget of 40 ms is a strong reference, because it was matched to the mean computation of AnytimeSwarm on development data. Its weakness lies in the allocation across fleet scales. AnytimeSwarm-R used 24 ms per call on the small scale and 75 ms on the large scale. On the large scale, it served +0.59 [+0.34, +0.82] pp more reward than the budget of 40 ms and was on a par with the full budget (−0.06 [−0.38, +0.26] pp). On the small scale, it served +1.79 [+0.65, +3.00] pp more than the full budget and was on a par with the budget of 40 ms (+0.33 [−0.40, +1.09] pp). A single fixed budget is therefore either too long for small fleets or too short for large ones, whereas the adaptive rules find a suitable amount of computation on each scale without tuning. The advantage over ALNS with 40 ms was +0.43 [+0.11, +0.76] pp for $\kappa_B = 10$ and +0.20 [−0.19, +0.61] pp for $\kappa_B = 50$, and +0.36 [+0.01, +0.73] pp under low and +0.27 [−0.13, +0.66] pp under mid disturbances.

S6.2 Replanning triggers

The trigger mattered more than the stopping rule. Replanning every 60 s in addition to the events raised the number of planning calls from 6.8 to 24.2 per mission and the utility of every method, by +1.29 pp to +1.56 pp, with all intervals above zero (+1.56 [+1.26, +1.84] pp for AnytimeSwarm-R, +1.47 [+1.19, +1.75] pp for ALNS with 40 ms). The third

hypothesis therefore holds. Periodic replanning probably helps because it corrects the accumulated deviations of the executed routes, which the event trigger only notices at the next event. The advantage of AnytimeSwarm-R over ALNS with 40 ms was +0.27 [−0.07, +0.62] pp with the event trigger and +0.36 [+0.03, +0.70] pp with the periodic trigger. The two values are close, and the protocol did not test their difference.

S6.3 Scalability to 40 UAVs

With 40 UAVs and 280 tasks, 90 held-out missions were run under mid disturbances in 4 configurations of latency level and latency factor. All five methods reached between 68.82% and 69.23%. AnytimeSwarm-R reached 69.16% with 127 ms per call. Its pooled difference was +0.02 [−0.26, +0.30] pp to ALNS with 180 ms, +0.34 [+0.05, +0.62] pp to AnytimeSwarm and −0.07 [−0.30, +0.16] pp to the stagnation rule. After Holm correction over the four comparisons of the planned analysis, none of them is significant ($p_{\text{Holm}} = 0.086$ for AnytimeSwarm), so the fifth hypothesis is not supported either. AnytimeSwarm-R again needed less computation than AnytimeSwarm (−9.0 [−11.7, −6.2] ms per call). The stagnation rule stopped earliest, after 100 ms, and was significantly better than AnytimeSwarm-R in 1 of the 4 configurations, which had no latency and $\kappa_B = 50$. On this scale, the rules used between 100 and 250 ms per call, and utility hardly changed within this range of computation. The equal-computation advantage that AnytimeSwarm showed with 40 UAVs under exact execution (Table S3) therefore did not translate into a utility gain under uncertain execution.

S6.4 Sensitivity analyses

The sensitivity runs use the held-out missions of the primary matrix without latency and are descriptive. They vary the execution level beyond the primary matrix, the latency factor and the trigger in 8 configurations, and the error model in 4 further configurations.

Under high disturbances, which lie outside the range of the primary matrix but within that of the fitted error models, utility fell to about 48.93% for all methods. The combination of high disturbances and a slow planner ($\kappa_B = 50$) gave the largest advantage of the risk-aware cost in this study. AnytimeSwarm-R served +0.96 [+0.29, +1.66] pp more reward than AnytimeSwarm and used 49 ms instead of 58 ms per call. Its difference to ALNS with 40 ms was +0.14 [−0.53, +0.84] pp and to the stagnation rule +0.43 [−0.33, +1.21] pp, so the gain over AnytimeSwarm brought it to the level of the baselines rather than beyond them. With $\kappa_B = 1$, planning time hardly delays the plan, both delay costs are close to zero because they scale with the shift $\kappa\tau$, and the two rules behave alike. AnytimeSwarm-R then differed from AnytimeSwarm by +0.01 [−0.13, +0.15] pp and −0.5 ms per call.

Replanning when a UAV deviated by more than 30 s from its planned finish time raised the number of calls from 6.8 to 13.2 per mission and the utility by +0.86 [+0.58, +1.14] pp for AnytimeSwarm-R and +0.86 [+0.54, +1.18] pp for ALNS with 40 ms. Like the periodic trigger, it helped both stopping rules by a similar amount. Under the deviation trigger, the difference of AnytimeSwarm-R was +0.10 [−0.30, +0.52] pp to ALNS with 40 ms and +0.29 [−0.05, +0.65] pp to AnytimeSwarm.

The utility of AnytimeSwarm-R was insensitive to the error model. With the error model of the low, mid or high level under low and mid disturbances, its difference to AnytimeSwarm ranged between +0.07 [−0.31, +0.47] pp and +0.18 [−0.18, +0.55] pp, the correctly specified model was not better than a wrong one, and the mean utilities of all versions lay within 0.11 pp. Setting the error model to zero gave −0.13 [−0.32, +0.03] pp. The four misspecification configurations also belong to the primary matrix, so the correctly specified method was run twice on the same missions. The two runs differ only in the measured planning times, because the stopping rule reads a wall clock. They gave identical utility in 73% of the missions, and the mean utility of a method differed by up to 0.12 pp between the runs. Differences of this size between stopping rules should therefore not be read as systematic.

S6.5 Tier C: PX4 software-in-the-loop with Gazebo

All 120 held-out flights of tier C were completed, that is, 30 missions with four methods each. Two runs did not reach their altitude within 90 s and were repeated as specified. A further 4 runs had a real-time factor between 1.43 and 1.52, below the validity range of 1.6 to 2.2 stated in the protocol. The protocol did not specify how to treat such runs, so they were repeated once with the same seed, like a technical failure, and the original runs were kept. The repeated runs reached the same utility as the original ones, and the comparisons below are identical with either set (−0.51 [−3.24, +2.18] pp and +0.48 [−2.72, +3.46] pp with the original runs). The real-time factor of the analysed runs was 1.95 on average and at least 1.67. A released plan reached all vehicles after 0.152 s of simulation time on average, which is included in the expected latency of AnytimeSwarm-R.

Table S5 summarises the results. AnytimeSwarm-R reached a utility of 54.18%, against 54.69% for ALNS with 40 ms, 53.70% for AnytimeSwarm and 51.88% for the full budget. The paired differences were −0.51 [−3.24, +2.18] pp to ALNS with 40 ms and +0.48 [−2.72, +3.46] pp to AnytimeSwarm, with Holm-adjusted p-values of 1.000 and 1.000. The fourth hypothesis is therefore not supported. With 30 missions, the intervals are much wider than in tier B and cannot resolve differences of the size seen there. Two effects of tier B reappear in tier C. The full budget again had

Table S5: Study 1: software-in-the-loop results.

Method	Utility C (%)	Lost to delay	Skipped	ms/call	Utility B (%)
ALNS, 250 ms	51.88	0.43	0.57	250.1	50.17
ALNS, 40 ms	54.69	0.03	0.50	40.1	51.72
AnytimeSwarm	53.70	0.03	0.43	47.4	51.90
AnytimeSwarm-R	54.18	0.00	0.37	34.7	52.55

30 held-out missions per method, mid disturbances, latency factor 50. Utility, tasks lost to late plans and skipped at departure per mission, planning time per call, and tier B utility on the same rescaled missions.

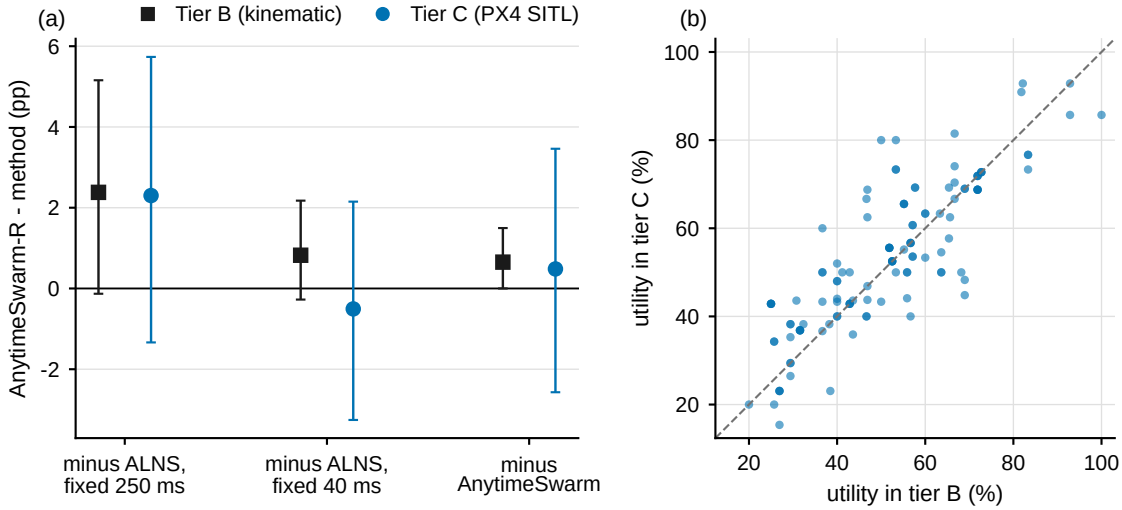


Figure S3: Study 1, tier B against tier C on the same 30 missions. (a) AnytimeSwarm-R minus each baseline, with 95% intervals. (b) Utility of every run in tier C against the same run in tier B; the dashed line marks equality.

the lowest mean utility, although its interval includes zero (+2.30 [−1.32, +5.71] pp to AnytimeSwarm-R), and it was the only method that lost a noticeable number of tasks to late plans, 0.43 per mission against at most 0.03 for the others. AnytimeSwarm-R again computed least among the adaptive rules, with 34.7 ms per call against 47.4 ms for AnytimeSwarm, a reduction of 27% (−12.7 [−18.8, −7.2] ms).

The same 30 missions were also run in tier B with the rescaled parameters of tier C. Tier B predicted the outcome of a flight well. The utility of a run in tier C correlated with that of the same run in tier B with $r = 0.84$ (120 runs), and the mean utility per mission with $r = 0.89$ (Fig. S3b). Tier C served slightly more reward on average, 53.61% against 51.58%, so the kinematic model of tier B was, if anything, slightly pessimistic for these missions. The small differences between the methods were not predicted as well. The method means of the two tiers had a Kendall rank correlation of 0.33, and the sign of the per-mission difference between AnytimeSwarm-R and ALNS with 40 ms agreed in 40% of the missions, counting ties as agreement only when both differences are zero. Part of the per-run correlation is by construction, because both tiers draw the wind, speed and service disturbances from the same random numbers, and much of it reflects the different difficulty of the missions. The correlation was first computed with an erroneous pairing of runs in the analysis script, which was corrected after the results had been seen; the hypothesis tests were not affected. In tier B, the advantage of AnytimeSwarm-R on these missions was +0.82 [−0.27, +2.16] pp over ALNS with 40 ms and +0.65 [+0.00, +1.50] pp over AnytimeSwarm (Fig. S3a). Both tiers thus agree that the methods differ by at most a few percentage points on this scale, and that the full budget is the weakest search method, while the order of the three short-computation methods is within the noise of both tiers.

S7 Study 2: regimes and software-in-the-loop details

Table S6 lists the results of all methods and ablations of study 2, and Fig. S4 shows their paired differences together with the computation of every method.

Over all configurations, LTCC without fallback served 49% fewer tasks late than ALNS with 40 ms. The stopping rules hardly mattered on the nominal model, with utilities between 49.28% and 49.66% although their computation ranged from 11.2 to 58.1 ms per call. Their value lay in avoiding long budgets on slow planners: with a latency factor of 200, the full budget of 250 ms lost 23.42 tasks per mission to late plans. Periodic replanning helped every method on the nominal model, but added only 0.22 pp with the effective deadlines.

Table S6: Study 2: tier B results of all methods.

Method	Utility (%)	Late	Lost	Calls	ms/call	LTCC minus method (pp)
Conventional planner	47.71	10.31	3.32	6.8	40.2	+4.44 [+3.82, +5.05]
Regret-2 repair (no search)	47.95	11.19	2.34	23.2	0.5	+4.21 [+3.57, +4.84]
ALNS, fixed 250 ms [9]	46.23	9.78	11.58	22.9	250.1	+5.92 [+5.41, +6.44]
ALNS, fixed 40 ms [9]	49.28	11.22	3.58	23.3	40.1	+2.87 [+2.29, +3.45], p <0.001*
ALNS, size-scaled budget	49.58	11.18	3.75	23.3	17.7	+2.57 [+1.99, +3.17], p <0.001*
ALNS, stagnation stop	49.04	10.98	5.50	23.2	55.8	+3.11 [+2.54, +3.69]
Performance profile [10, 11]	49.54	11.24	3.52	23.2	30.2	+2.62 [+2.04, +3.19], p <0.001*
Online performance prediction [5]	49.45	11.17	2.75	23.2	11.2	+2.70 [+2.13, +3.28], p <0.001*
AnytimeSwarm	49.31	11.17	3.89	23.2	58.1	+2.84 [+2.29, +3.40]
AnytimeSwarm-R	49.66	11.21	3.73	23.2	50.9	+2.49 [+1.92, +3.06], p <0.001*
Constant buffer (60 s) [12]	50.81	6.53	0.00	23.4	40.1	+1.35 [+0.93, +1.77], p <0.001*
LTCC, no fallback	52.15	5.72	0.00	23.3	61.0	-
Ablations of LTCC						
LTCC, event trigger only	51.93	5.85	0.00	6.8	57.8	+0.22 [+0.06, +0.38]
LTCC, fixed 40 ms	51.93	5.74	0.00	23.4	40.1	+0.22 [+0.07, +0.37]
LTCC, mean correction only	51.97	6.84	0.00	23.3	58.8	+0.18 [−0.06, +0.41]

270 missions in 12 configurations per method. Late: tasks served late; Lost: tasks lost because a plan arrived too late; Calls: planning calls per mission. The last column gives LTCC minus each method, pooled over configurations (pp, 95% paired bootstrap interval; Holm-adjusted p for the six primary comparisons, marked *).

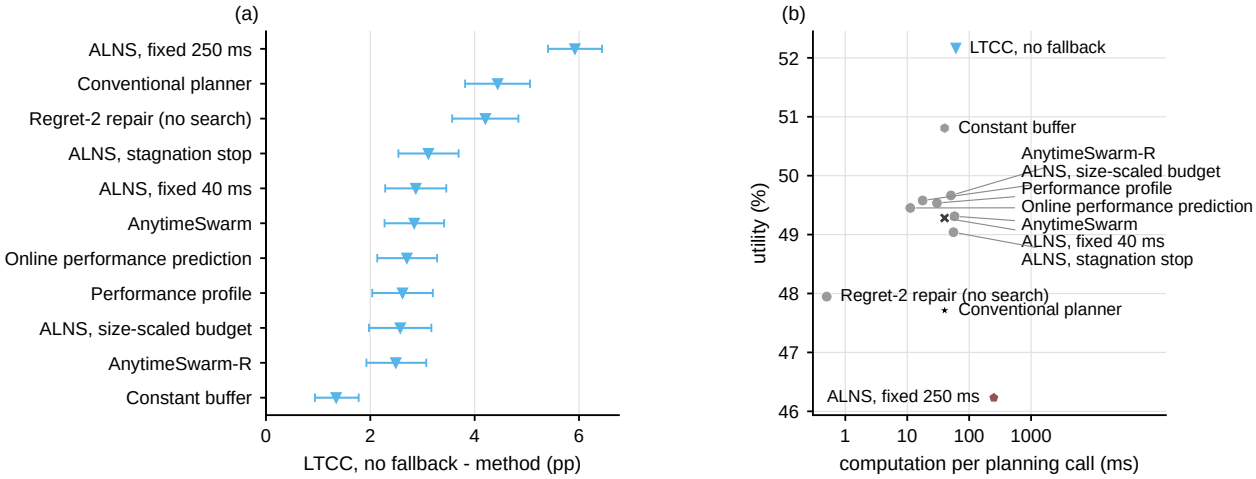


Figure S4: Study 2, tier B. (a) Utility of LTCC without fallback minus each method, with 95% intervals over 270 missions. (b) Mean utility against mean computation per call.

S7.1 Regimes

Table S7 and Fig. S5 break the utility down by disturbance level, time-window factor and latency factor. Per configuration and without correction for multiple testing, LTCC without fallback was better than ALNS with 40 ms in 8 of the 12 configurations and worse in 2, better than the constant buffer in 9 and worse in 3, and better than the conventional planner in 10 and worse in 0. The advantage over ALNS with 40 ms was +2.58 [+1.93, +3.22] pp, +2.71 [+2.10, +3.33] pp and +3.33 [+2.74, +3.91] pp for κ_B = 10, 50 and 200, and the advantage over online prediction stayed between +2.76 [+2.15, +3.36] pp and +2.57 [+1.97, +3.16] pp.

The advantage was largest with the original windows. Under high disturbances, LTCC without fallback served +6.22 [+5.17, +7.29] pp more reward than ALNS with 40 ms and +3.03 [+2.01, +4.11] pp more than the constant buffer. The losses all occurred under high disturbances with halved windows. In this regime, LTCC without fallback reached 33.38%, −2.01 [−2.80, −1.21] pp relative to the constant buffer and −1.57 [−2.52, −0.64] pp relative to ALNS with 40 ms. It served 24.3 tasks per mission, of which 6.2 late, against 25.3 and 8.4 for the constant buffer. The chance-constrained model was thus too cautious when the windows were short compared with the predicted finish-time error. The effective deadline of many tasks then probably falls before the earliest possible arrival, so the planner gives up tasks that a less cautious plan still serves in time. The value of z was selected per disturbance level on development missions, averaged over both window widths. Selecting it also per window width, or adapting it online, is a natural extension.

Table S7: Study 2: utility (%) per regime.

	mid, windows ×1			mid, windows ×0.5			high, windows ×1			high, windows ×0.5		
Method	10	50	200	10	50	200	10	50	200	10	50	200
Constant buffer (60 s)	70.1	70.1	69.1	45.6	45.4	44.5	53.1	53.0	52.7	35.7	35.5	34.9
ALNS, fixed 40 ms	67.0	66.5	65.8	47.1	46.5	44.4	49.9	49.6	49.7	35.4	35.3	34.2
ALNS, fixed 250 ms	67.4	66.0	60.6	46.6	43.4	31.0	50.8	50.3	46.0	35.2	33.1	24.4
Conventional planner	65.6	65.5	63.9	45.4	44.9	42.7	48.7	48.6	47.5	34.1	33.6	32.2
LTCC, no fallback	72.1	71.8	71.0	47.8	47.7	47.4	56.3	55.9	55.7	33.5	33.3	33.3

Disturbance level, time-window factor and latency factor (10, 50, 200); tier B held-out missions.

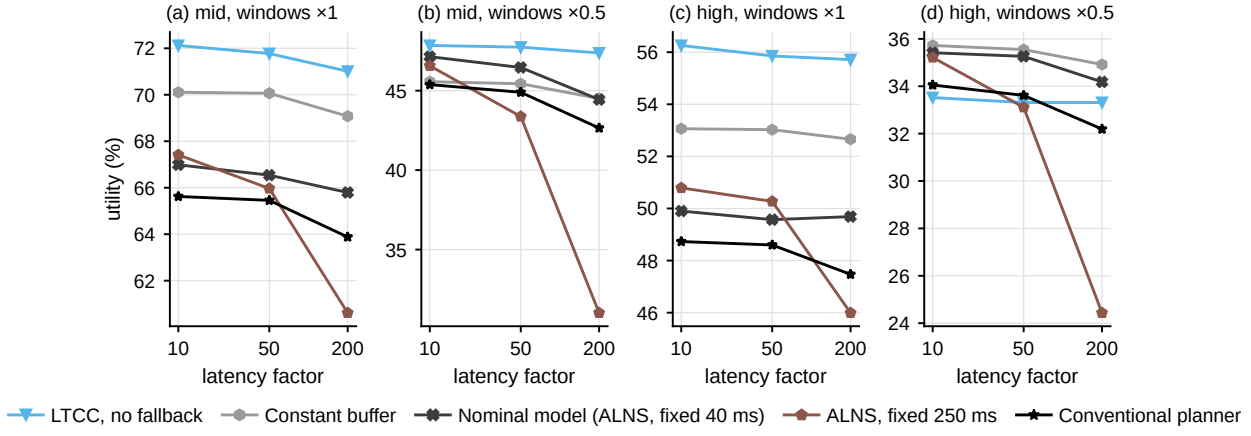


Figure S5: Study 2: utility per regime against the latency factor, mean over 270 missions per point.

S7.2 Tier C: PX4 software-in-the-loop with Gazebo

All 120 held-out flights of study 2 were completed, that is, 30 missions with four methods each. One run failed to start and was repeated. As specified in the protocol, 46 runs with a real-time factor outside 1.6 to 2.2 were repeated once with the same seed, and the original runs were kept. The laptop was also used for other work during the campaign, and many repeats ran slower than the original runs. Of the 120 analysed runs, 77 lie in the validity range. The real-time factor was 1.64 on average, with quartiles of 1.41-1.95 and a minimum of 0.83. Two sensitivity analyses that were not planned in the protocol therefore repeat the comparisons with the original runs instead of the repeats, and with only those pairs of runs that both lie in the validity range.

Table S8 summarises the results. LTCC without fallback reached a utility of 59.89%, against 60.15% for ALNS with 40 ms, 57.26% for the constant buffer and 58.61% for the conventional planner. The paired differences were −0.25 [−4.02, +3.57] pp to ALNS with 40 ms and +1.28 [−2.20, +4.64] pp to the conventional planner, with Holm-adjusted p-values of 0.942 and 0.942. The fourth hypothesis is therefore not supported. The secondary comparison with the constant buffer gave +2.64 [−0.01, +5.58] pp. The sensitivity analyses lead to the same conclusion. With the original runs, the differences were −1.04 [−4.83, +2.77] pp and −0.02 [−3.66, +3.47] pp. With valid runs only, they were −2.02 [−6.25, +2.43] pp on 16 missions and −1.52 [−6.32, +3.37] pp on 14 missions. No interval in the three analyses excludes zero.

The mechanism seen in tier B reappears in the counts of late and skipped tasks. LTCC without fallback served 0.90 tasks late per mission, against 1.20 for ALNS with 40 ms, which is 25% fewer. It skipped no task at departure, whereas ALNS with 40 ms skipped 0.23 and the conventional planner 0.57 tasks per mission. It computed 44.4 ms per call, against 40.2 ms for the fixed budget. The conventional planner called the search 6.3 times per mission instead of about 21.9, as in tier B.

The same 30 missions were also run in tier B with the rescaled parameters of tier C. As in study 1, tier B predicted the utility of a flight closely, with a correlation of $r = 0.86$ over 120 runs. On these missions, however, even tier B did not resolve the differences between the methods (Fig. S6). Its estimates were +2.82 [−1.20, +6.62] pp to ALNS with 40 ms, +1.90 [−0.59, +4.65] pp to the constant buffer and +1.80 [−2.31, +5.65] pp to the conventional planner. The tier C intervals overlap them. The per-mission difference to ALNS with 40 ms had a standard deviation of 10.9 pp in tier C. Confirming the tier B estimate of 2.8 pp on these missions with a power of 80% would require about 118 missions per method. Tier C thus shows that the proposed replanner runs on an autopilot and a physics engine with the same computation and fewer late tasks, but it cannot confirm the size of the gain measured in tier B.

Table S8: Study 2: software-in-the-loop results.

Method	Utility C (%)	Late	Skipped	Calls	ms/call	Utility B (%)
ALNS, fixed 40 ms	60.15	1.20	0.23	20.9	40.2	58.67
Constant buffer (60 s)	57.26	0.83	0.00	21.0	40.2	59.60
Conventional planner	58.61	1.37	0.57	6.3	40.2	59.70
LTCC, no fallback	59.89	0.90	0.00	21.9	44.4	61.50

30 held-out missions per method, mid disturbances, latency factor 50. Utility, tasks served late, tasks skipped at departure, planning calls and computation per call, and tier B utility on the same rescaled missions.

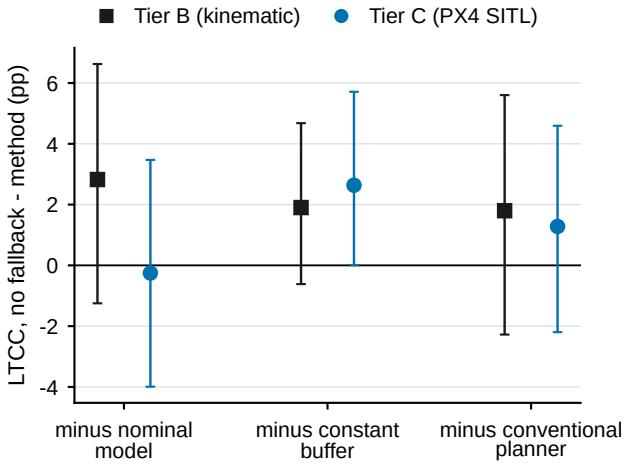


Figure S6: Study 2, tier B against tier C on the same 30 missions: LTCC without fallback minus each baseline, with 95% intervals.

S8 Study 3: regimes

Table S9 lists the results of study 3, and Fig. S7 shows its paired differences per regime. Over all configurations, LTCC with nominal fallback served +4.61 [+4.15, +5.06] and +3.59 [+3.17, +4.02] pp more reward than ALNS with 40 ms and the constant buffer (H7, H8). The fallback has a price in punctuality, since a fallback task is planned against its true deadline and is often served late. It served 8.33 tasks late per mission, 46% more than LTCC without fallback.

S9 Study 4: regimes and reliability of the error model

Fig. S8 summarises the replication of the main comparisons on the held-out seeds of studies 2, 3 and 4.

Fig. S9 shows the paired differences of study 4 per regime. All three advantages of LTCC, over the constant buffer, the nominal model and the conventional planner, held in each of the four regimes.

Fig. S10 groups the tasks that were served in study 4 by the probability of meeting the deadline that the error model of their disturbance level predicted at the last published plan, and compares it with the share served on time. The plans of LTCC and of the nominal model are evaluated with the same error model. Under mid disturbances, the share served on time was 88.8% for LTCC, against a mean predicted probability of 90.2%. Under high disturbances, it was 76.9% against 71.3%. Among the tasks with a predicted probability of at least one half, the shares were 92.8% and 88.1%, against predicted means of 95.2% and 85.0%. The model was thus slightly optimistic under mid and conservative under high disturbances, and it ranked the tasks by their risk in both cases.

S10 Study 5: cost of a late arrival

Fig. S11 scores the held-out runs of studies 3 and 4 again with a cost of λ mean task rewards for every late arrival, $u_\lambda = U - \lambda\, n_{\text{late}}/n$. The analysis was done after study 4C, was not planned in a protocol and used no new runs. The fallback of study 3 raised the number of tasks served late from 5.70 to 8.33 per mission, and the fallback within moment propagation from 1.62 to 7.36. A comparison changes sign at the late cost $\lambda^* = \Delta U/\Delta(n_{\text{late}}/n)$. For the fallback, λ^* was 0.5 in study 3 and 1.1 within moment propagation. In tier C, it was 1.5, with a difference of +3.50 [−1.43, +8.41] pp at $\lambda = 1$. The advantage over the constant buffer shrank with λ and vanished at 2.3, because the buffer of 60 s served fewer tasks late than LTCC with nominal fallback (6.52 against 7.54 per mission). Against the proportional

Table S9: Study 3: utility (%) per method and regime.

		Mid		High			
Method	All	×1	×0.5	×1	×0.5	Late	ms/call
Constant buffer (60 s)	50.80	69.33	45.48	52.48	35.92	6.62	40.1
ALNS, fixed 40 ms	49.79	67.38	46.05	50.10	35.61	11.18	40.1
Conventional planner	48.20	65.83	44.46	48.61	33.91	10.27	40.1
LTCC, no fallback	52.42	71.56	47.88	56.28	33.97	5.70	57.4
LTCC, nominal fallback	54.40	72.08	49.89	56.39	39.22	8.33	55.0

Regimes: disturbance level and time-window factor, averaged over the latency factors. Late: tasks served late per mission. 270 missions, 12 configurations.

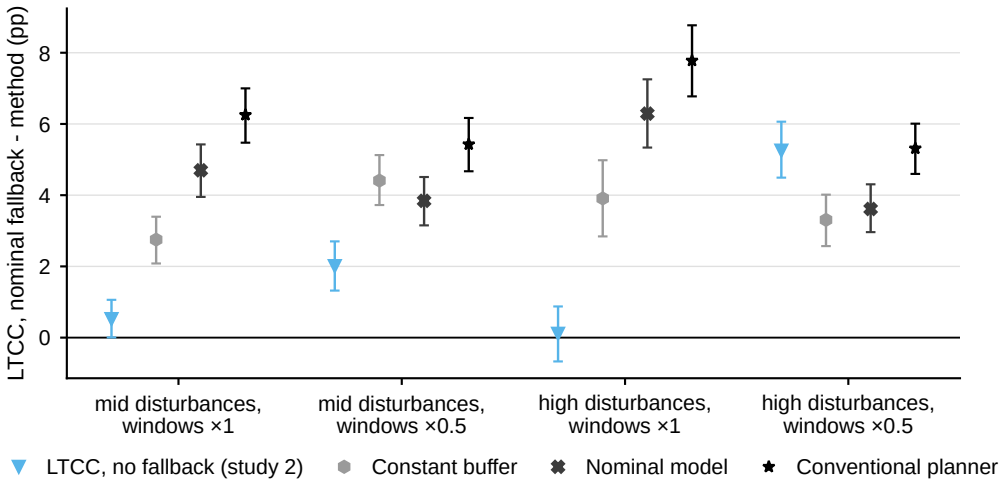


Figure S7: Study 3: utility of LTCC with nominal fallback minus each method per regime, with 95% intervals over 270 missions.

buffer, the proposed model with fallback stayed behind at every λ, by 1.00 pp at $\lambda = 1$ (-1.00 [-1.50, -0.50]). Study 5 (Section 6.4 of the main paper) was designed from these results.

S11 Calibration of the predicted probability

Every tier B run since study 4 bins its executed tasks by the probability of meeting the deadline that the error model of the disturbance level predicted at the last published plan, and counts how many of them met it. From these bins, Fig. S12 shows the reliability of the prediction for LTCC in studies 4-6 and, in its right panel, under the shifted execution errors of study 7b. Over studies 4-6, the expected calibration error, ECE $= \sum_i |k_i - p_i|/N$ with k_i the tasks on time and p_i the sum of the predicted probabilities in decile i, ranged from 2.3 to 6.1 pp. The upper tail, a predicted probability of at least 0.8, is where a chance constraint acts. There, 94.8% of the tasks were served on time under mid disturbances in study 5, against a predicted 97.3%, and 94.3% under high disturbances, against 93.6%. Below the tail, the model was conservative under high disturbances, where it predicted a lower probability than was observed, and slightly optimistic under mid disturbances. Under the shifted errors of study 7b, the ECE was 4.5 pp for the calibrated Gaussian errors, 4.5 pp for Student-t errors, 3.6 pp with rare long delays, 4.6 pp with a speed bias per UAV and 4.4 pp with Pareto latency, with Brier scores of 0.118, 0.114, 0.129, 0.122 and 0.118. In the upper tail, the share on time under these conditions was 94.0, 94.8, 92.6, 93.2 and 94.0%, against predicted means of 93.7, 93.7, 93.6, 93.7 and 93.7%.

S12 Sensitivity to the resampling unit

The primary tests resample missions. The nine missions of one seed have independent scenarios but share the execution disturbances, which are keyed to the seed, the UAV and the task. Table S10 repeats every tier B primary test of studies 2-7 with a paired bootstrap over the 30 seeds. The intervals were 0.69 to 1.18 times as wide as over missions (median 0.97). They were mostly narrower, because every seed contains one mission of each fleet scale and event profile, so that resampling seeds removes the variation between these strata. 1 of the 36 outcomes changed. In H9, the advantage of the proportional buffer over LTCC, which had missed significance over missions, became significant over seeds (interval

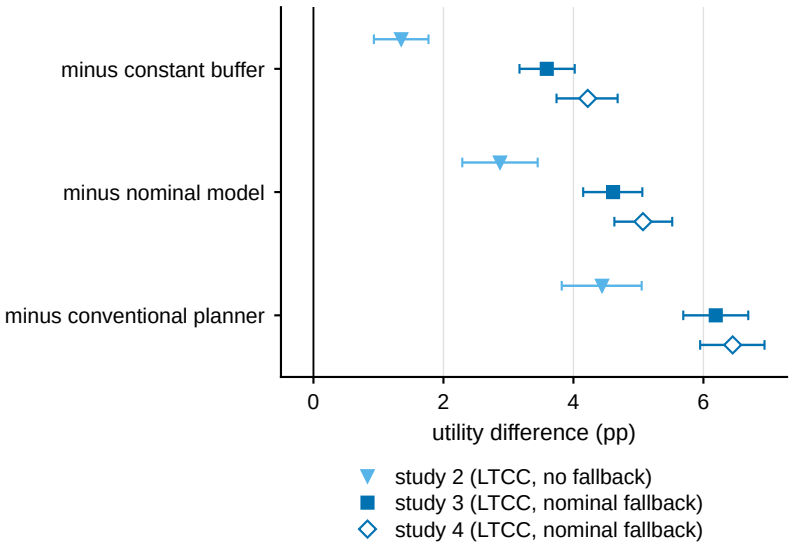


Figure S8: Replication on three independent sets of held-out seeds: utility of LTCC minus each method, with 95% intervals over 270 missions per study.

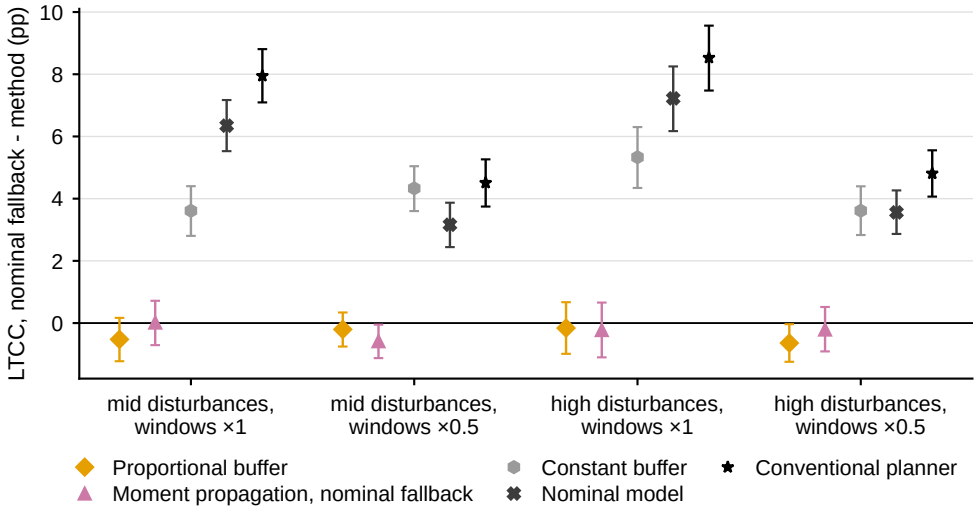


Figure S9: Study 4: utility of LTCC with nominal fallback minus each planning model per regime, with 95% intervals over 270 missions.

−0.69 to −0.07 pp for LTCC minus the buffer, Holm-adjusted $p = 0.032$). The tier C hypotheses have one mission per seed and are not affected.

S13 Study 7: further results

Study 7a ran the cost-aware fallback with both cost models for $\lambda \in \{0.5, 1, 2\}$ on seeds 13000-13029. The competitors were selected on development missions per score, late cost and disturbance level. For u_λ, the proportional buffer was prop=0.45, fb nominal / prop=0.75, fb nominal, prop=0.3, fb none / prop=0.45, fb none and prop=0.45, fb none / prop=0.75, fb none for $\lambda = 0.5$, 1 and 2 (mid / high disturbances), the same selection as in study 5. For u^w_λ, it was prop=0.45, fb nominal / prop=0.75, fb nominal, prop=0.3, fb none / prop=0.6, fb none and prop=0.45, fb none / prop=0.75, fb none. Table S11 gives the scores of the main methods.

Against the nominal fallback, the fixed-cost rule scored +2.45 [+1.95, +2.96] pp more with u_1 and +8.06 [+7.31, +8.84] pp more with u_2; the proportional rule scored +2.37 [+1.79, +2.96] pp more with u^w_1. By regime at $\lambda = 1$, the fixed-cost rule minus the tuned buffer with u_1 was −0.90 [−1.84, +0.04] pp under mid disturbances with the original windows, +1.71 [+0.89, +2.53] pp with halved windows, and +1.92 [+0.62, +3.24] and +2.62 [+1.54, +3.70] pp under high disturbances with the original and the halved windows. As in study 5, the advantage came from high disturbances and short windows, and with mid disturbances and the original windows the tuned buffer was ahead.

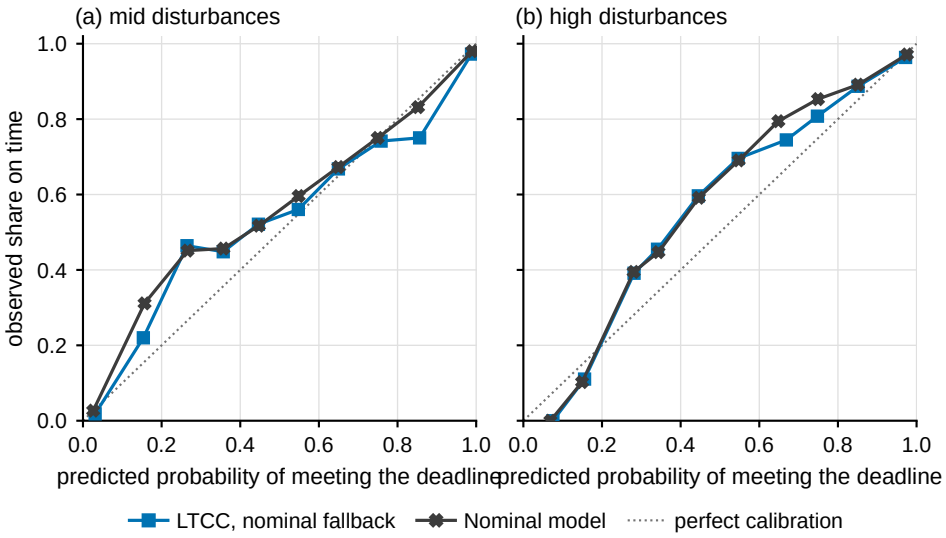


Figure S10: Study 4: share of served tasks on time against the predicted probability, for LTCC with nominal fallback and the nominal model. The dotted line marks perfect calibration.

The fixed-cost rule assumes a cost of $\lambda\bar{w}$ per late arrival with the mean reward $\bar{w} = 2.1$ of the generator, whereas u_λ charges $\lambda R/n$ with the realised mean reward of the mission, which ranged from 1.64 to 2.76 over the held-out missions. Table S12 repeats the comparisons of the fixed-cost rule with $u'_\lambda = U - \lambda\bar{w}\, n_{\text{late}}/R$, which charges exactly $\lambda\bar{w}$. All differences changed by at most a few hundredths of a point, and no conclusion changed. Using the realised R/n in the rule itself is not possible during a mission, because R includes the tasks that have not yet arrived.

Study 7b ran on seeds 14000-14029 under high disturbances. Table S13 gives the utility and the score u_1 of every method under the calibrated Gaussian errors and the four departures. The mean finish-time error of the proposed model with the cost-aware fallback was 78.8, 77.3, 88.3, 84.1 and 78.7 s, and its 95th percentile 330.3, 316.5, 348.1, 356.1 and 328.6 s. On u_1, the fixed-cost rule scored 0.38 to 0.70 pp above the proportional rule, and moment propagation with the cost-aware fallback 0.43 to 0.63 pp above the closed form, with every interval including zero in both cases. Moment propagation served less utility than the closed form under every condition.

References

[1] P. McCullagh, J. A. Nelder, Generalized Linear Models, 2nd Edition, Chapman and Hall, London, 1989. doi:10.1007/978-1-4899-3242-6.

[2] M. W. P. Savelsbergh, The vehicle routing problem with time windows: Minimizing route duration, ORSA J. Comput. 4 (2) (1992) 146-154. doi:10.1287/ijoc.4.2.146.

[3] L. Perron, V. Furnon, OR-Tools, https://developers.google.com/optimization, version 9.15 (2026).

[4] C. Voudouris, E. Tsang, Guided local search and its application to the traveling salesman problem, Eur. J. Oper. Res. 113 (2) (1999) 469-499. doi:10.1016/S0377-2217(98)00099-X.

[5] J. Svegliato, K. H. Wray, S. Zilberstein, Meta-level control of anytime algorithms with online performance prediction, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-18), IJCAI Organization, 2018, pp. 1499-1505. doi:10.24963/ijcai.2018/208.

[6] C. E. Clark, The greatest of a finite set of random variables, Oper. Res. 9 (2) (1961) 145-162. doi:10.1287/opre.9.2.145.

[7] L. B. Johnson, H.-L. Choi, S. S. Ponda, J. P. How, Allowing non-submodular score functions in distributed task allocation, in: Proceedings of the 51st IEEE Conference on Decision and Control (CDC), 2012, pp. 4702-4708. doi:10.1109/CDC.2012.6425867.

[8] T. Fawcett, An introduction to ROC analysis, Pattern Recognit. Lett. 27 (8) (2006) 861-874. doi:10.1016/j.patrec.2005.10.010.

[9] S. Ropke, D. Pisinger, An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows, Transp. Sci. 40 (4) (2006) 455-472. doi:10.1287/trsc.1050.0135.

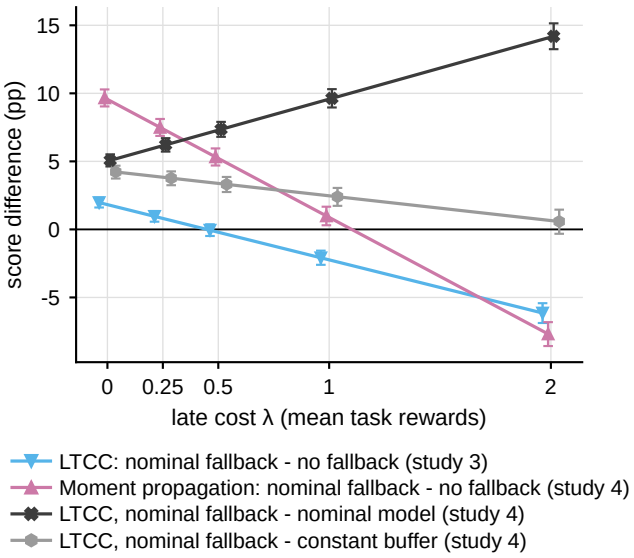


Figure S11: Studies 3 and 4 re-scored with a late cost of λ mean task rewards: score differences with 95% intervals over 270 missions (exploratory, no new runs).

[10] M. Boddy, T. L. Dean, Deliberation scheduling for problem solving in time-constrained environments, Artif. Intell. 67 (2) (1994) 245-285. doi:10.1016/0004-3702(94)90054-X.

[11] S. Zilberstein, Using anytime algorithms in intelligent systems, AI Mag. 17 (3) (1996) 73-83. doi:10.1609/aimag.v17i3.1232.

[12] O. Jabali, R. Leus, T. Van Woensel, T. de Kok, Self-imposed time windows in vehicle routing problems, OR Spectrum 37 (2) (2015) 331-352. doi:10.1007/s00291-013-0348-1.

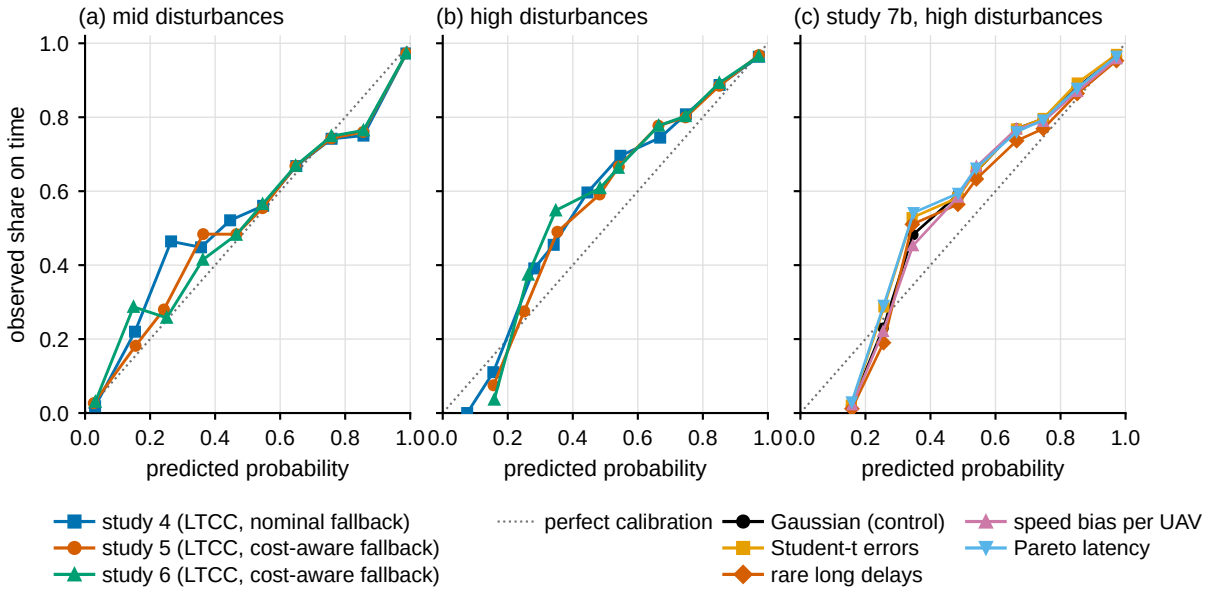


Figure S12: Reliability of the predicted probability of meeting the deadline for LTCC. (a) Mid and (b) high disturbances in studies 4-6. (c) Study 7b with the calibrated errors and four departures from the error model. The dotted line marks perfect calibration.

Table S10: Sensitivity of the tier B primary tests to the resampling unit.

H	Comparison	Diff.	Mission interval	p	Seed-cluster interval	p	Outcome (both)
H1	vs. ALNS 40 ms	+2.87	[+2.29, +3.45]	<0.001	[+2.25, +3.44]	<0.001	supported
H1	vs. size-scaled budget	+2.57	[+1.99, +3.17]	<0.001	[+1.93, +3.16]	<0.001	supported
H1	vs. online prediction	+2.70	[+2.13, +3.28]	<0.001	[+2.07, +3.28]	<0.001	supported
H1	vs. performance profile	+2.62	[+2.04, +3.19]	<0.001	[+2.03, +3.17]	<0.001	supported
H1	vs. AnytimeSwarm-R	+2.49	[+1.92, +3.06]	<0.001	[+1.90, +3.04]	<0.001	supported
H1	vs. constant buffer	+1.35	[+0.93, +1.77]	<0.001	[+1.02, +1.69]	<0.001	supported
H2	vs. conventional	+4.44	[+3.82, +5.05]	<0.001	[+3.78, +5.10]	<0.001	supported
H3	stopping rule vs. fixed 40 ms	+0.22	[+0.09, +0.35]	-	[+0.07, +0.37]	-	equivalent
H5	fallback vs. no fallback	+1.97	[+1.61, +2.33]	<0.001	[+1.63, +2.33]	<0.001	supported
H6	vs. buffer, high, ×0.5	+3.31	[+2.58, +4.03]	<0.001	[+2.73, +3.90]	<0.001	supported
H7	vs. ALNS 40 ms	+4.61	[+4.15, +5.06]	<0.001	[+4.17, +5.05]	<0.001	supported
H8	vs. constant buffer	+3.59	[+3.17, +4.02]	<0.001	[+3.25, +3.93]	<0.001	supported
H9	vs. proportional buffer	−0.38	[−0.72, −0.05]	0.050	[−0.69, −0.07]	0.032	not supported / reversed
H10	vs. moment propagation (NI)	−0.25	[−0.62, +0.12]	0.092	[−0.61, +0.11]	0.084	NI not shown
H11	MP: fallback vs. none	+9.66	[+9.04, +10.29]	<0.001	[+8.99, +10.36]	<0.001	supported
H12	vs. constant buffer	+4.22	[+3.74, +4.68]	<0.001	[+3.87, +4.59]	<0.001	supported
H13	vs. nominal model	+5.07	[+4.63, +5.52]	<0.001	[+4.63, +5.49]	<0.001	supported
H16	vs. nominal fallback	+2.42	[+1.81, +3.02]	<0.001	[+2.00, +2.83]	<0.001	supported
H17	vs. tuned prop. buffer (NI)	+0.90	[+0.38, +1.44]	<0.001	[+0.39, +1.40]	<0.001	NI shown
H18	vs. tuned z, no fallback (NI)	+1.78	[+1.25, +2.31]	<0.001	[+1.29, +2.28]	<0.001	NI shown
H19	vs. nominal model	+11.75	[+10.88, +12.57]	<0.001	[+11.12, +12.38]	<0.001	supported
H20	vs. CBBA	+12.77	[+11.91, +13.63]	<0.001	[+12.09, +13.45]	<0.001	supported
H21	CBBA on proposed model vs. CBBA	+12.12	[+11.31, +12.93]	<0.001	[+11.49, +12.72]	<0.001	supported
H22	MP: cost-aware vs. nominal fb	+2.71	[+2.13, +3.27]	<0.001	[+2.13, +3.26]	<0.001	supported
H23	MP: cost-aware vs. no fallback	+3.30	[+2.83, +3.79]	<0.001	[+2.90, +3.69]	<0.001	supported
H24	fixed-cost vs. proportional rule	+0.24	[−0.20, +0.67]	0.286	[−0.18, +0.66]	0.269	not supported
H25	fixed-cost rule vs. tuned buffer (NI)	+1.34	[+0.77, +1.90]	<0.001	[+0.76, +1.92]	<0.001	NI shown
H26	prop. rule vs. buffer tuned for u_1^w (NI)	+0.20	[−0.40, +0.79]	0.022	[−0.34, +0.74]	0.011	NI shown
H27	Student-t errors: nominal fb vs. nominal	+5.40	[+4.81, +5.98]	<0.001	[+4.83, +5.94]	<0.001	supported
H28	Student-t errors: cost-aware vs. nominal fb	+3.64	[+2.78, +4.52]	<0.001	[+2.83, +4.51]	<0.001	supported
H27	rare long delays: nominal fb vs. nominal	+5.67	[+5.04, +6.30]	<0.001	[+5.11, +6.24]	<0.001	supported
H28	rare long delays: cost-aware vs. nominal fb	+3.87	[+2.96, +4.78]	<0.001	[+3.07, +4.73]	<0.001	supported
H27	speed bias per UAV: nominal fb vs. nominal	+4.83	[+4.25, +5.42]	<0.001	[+4.21, +5.50]	<0.001	supported
H28	speed bias per UAV: cost-aware vs. nominal fb	+3.64	[+2.79, +4.50]	<0.001	[+2.76, +4.50]	<0.001	supported
H27	Pareto latency: nominal fb vs. nominal	+5.25	[+4.62, +5.88]	<0.001	[+4.60, +5.95]	<0.001	supported
H28	Pareto latency: cost-aware vs. nominal fb	+3.60	[+2.73, +4.48]	<0.001	[+2.90, +4.37]	<0.001	supported

Mission: paired bootstrap over 270 missions, as in the primary analyses. Seed cluster: paired bootstrap over the 30 seeds, each with its nine missions, which share the execution disturbances. Differences in pp, 95% intervals (90% for the equivalence test H3), Holm-adjusted p within each primary family; for non-inferiority tests, p is the one-sided p with a margin of 0.5 pp.

Table S11: Study 7a: mean score (%) of the two rules and of the tuned proportional buffer.

	u_λ (fixed cost)			u^w_λ (cost $\propto w_j$)		
Method	$\lambda = 0.5$	1	2	0.5	1	2
Fixed-cost rule	48.73	45.11	39.53	47.81	43.09	36.47
Proportional rule	48.59	44.87	38.68	47.99	43.96	37.72
Tuned proportional buffer	48.56	43.78	39.43	48.15	43.76	39.08

Table S12: Study 7a: fixed-cost rule under two definitions of the fixed cost.

λ	Comparison	u_λ	u'_λ	p
0.5	fixed-cost vs. proportional rule	+0.14 [−0.20, +0.48]	+0.17 [−0.18, +0.51]	0.351
0.5	fixed-cost rule vs. tuned buffer	+0.17 [−0.27, +0.61]	+0.18 [−0.26, +0.62]	0.417
1	fixed-cost vs. proportional rule	+0.24 [−0.20, +0.67]	+0.26 [−0.17, +0.69]	0.245
1	fixed-cost rule vs. tuned buffer	+1.34 [+0.77, +1.90]	+1.35 [+0.79, +1.91]	<0.001
2	fixed-cost vs. proportional rule	+0.85 [+0.32, +1.39]	+0.87 [+0.34, +1.40]	0.002
2	fixed-cost rule vs. tuned buffer	+0.10 [−0.63, +0.82]	+0.10 [−0.62, +0.82]	0.770

u_λ charges a late arrival the realised mean reward R/n of the mission, and $u'_\lambda = U - \lambda\bar{w}\, n_{\text{late}}/R$ charges exactly the cost $\lambda\bar{w}$ that the rule assumes. Differences in pp with 95% paired bootstrap intervals over 270 missions; p for u'_λ.

Table S13: Study 7b: utility U / score u_1 (%) under errors outside the model.

Method	Gaussian (control)	Student-t errors	rare long delays	speed bias per UAV	Pareto latency
Nominal model	42.28 / 23.13	42.67 / 23.46	40.09 / 19.91	42.32 / 23.27	42.39 / 23.32
Constant buffer (60 s)	43.55 / 30.40	43.84 / 30.77	41.48 / 27.20	43.40 / 30.18	43.38 / 30.24
Proportional buffer	48.13 / 33.71	48.66 / 34.37	46.10 / 30.58	47.58 / 32.87	48.09 / 33.59
Moment propagation, cost-aware	42.22 / 37.14	42.68 / 37.95	40.69 / 34.76	41.83 / 36.54	42.18 / 37.15
LTCC, nominal fallback	47.63 / 33.13	48.07 / 33.67	45.76 / 30.34	47.15 / 32.45	47.64 / 33.11
LTCC, cost-aware, proportional rule	44.93 / 36.71	45.33 / 37.31	43.35 / 34.21	44.54 / 36.08	44.93 / 36.72
LTCC, cost-aware, fixed-cost rule	45.75 / 37.41	45.97 / 37.69	43.95 / 34.71	45.27 / 36.68	45.62 / 37.24

Held-out seeds 14000-14029, high disturbances. The error model, the safety factor and every tuned parameter are those of the calibrated level.